FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

流批一体在字节跳动特征平台的实践

Practice of Flink Stream-Batch Unification in ByteDance recommendation system

胡家煊/字节跳动推荐架构数据研发工程师

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

#1 背景介绍

#2 特征平台架构

#3 流批一体实践与探索

#4 未来规划

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

#1 业务背景

业务场景



- 抖音推荐



- 头条、西瓜 推荐

平台架构演进

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

第一阶段

基于 MapReduce、Storm 搭建的各种 profile 服务；



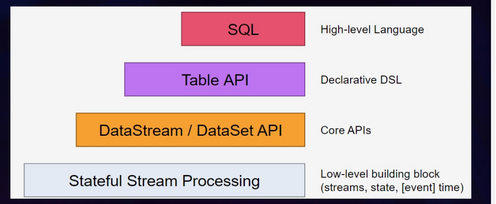
第二阶段

引入 Flink，拥抱大数据生态；



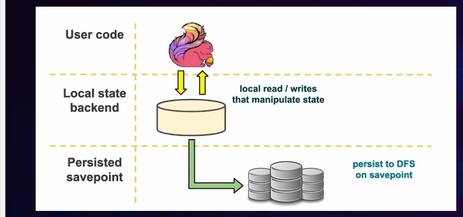
第三阶段

从 Native Flink 逐渐迁移至 Flink SQL；提升开发效率，降低运维成本；



第四阶段

基于 Flink SQL 与 State，构建以离线计算为核心的特征生产平台；

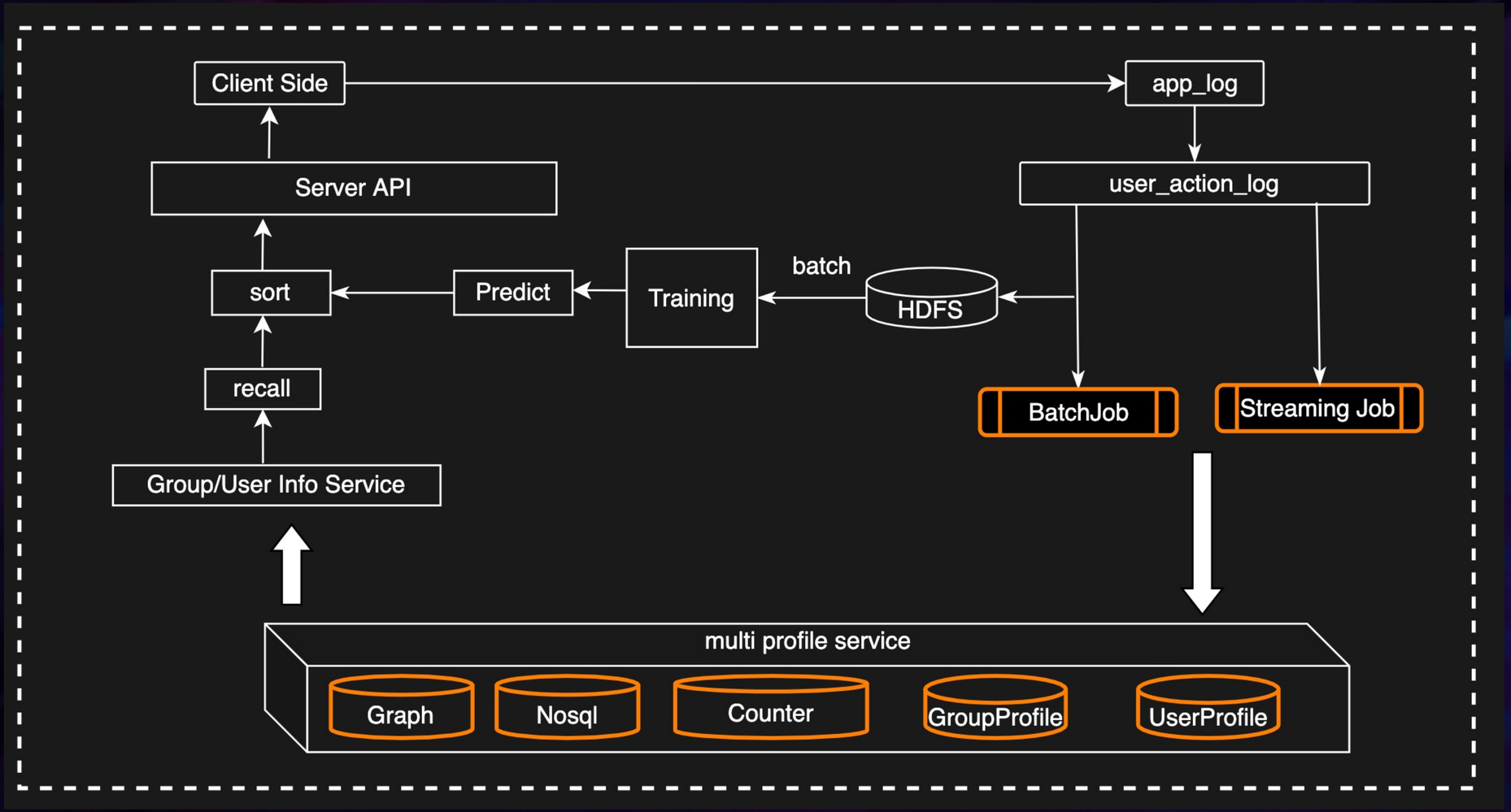


第五阶段

拥抱流批一体，探索新架构；



推荐系统数据流


Client Side
app_log
Server API
user_action_log
sort
Predict
Training
batch
HDFS
recall
BatchJob
Streaming Job
Group/User Info Service
multi profile service
Graph
Nosql
Counter
GroupProfile
UserProfile

特征分类

特征类型	特征主键	特征举例	特征来源
User侧	user_id、author_id	年龄、性别、行为计数	UserProfile
Group侧	item_id	类目、停留时长、播放量、完播次数等	GroupProfile、 Counter
Match类型	<source_id, target_id>	二跳统计、交互统计	图存储服务
Context类型	-	设备型号、位置信息、推出的时间	当前Context

核心问题

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

缺少特征全局视图

特征服务相对零散、边界比较模糊，可能存在冗余特征导致资源浪费。

离线任务游离

绝大部分特征服务以 C++ service 为核心，只提供读写接口，业务方自行维护离线任务。

Online Serving 性能

特征服务存储特征原始数据，请求时根据参数在线计算；部分场景读写扇出严重，影响延迟。

解决思路：以离线计算为核心

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

- 以离线任务为切入点，收敛特征生产入口；
- 使用 Flink SQL 解决特征计算的ETL逻辑；
- 使用 Flink State 存储特征计算所需的中间结果；
- 使用 Flink Timer 定时触发特征的计算，将计算好的结果更新到 KV Storage；

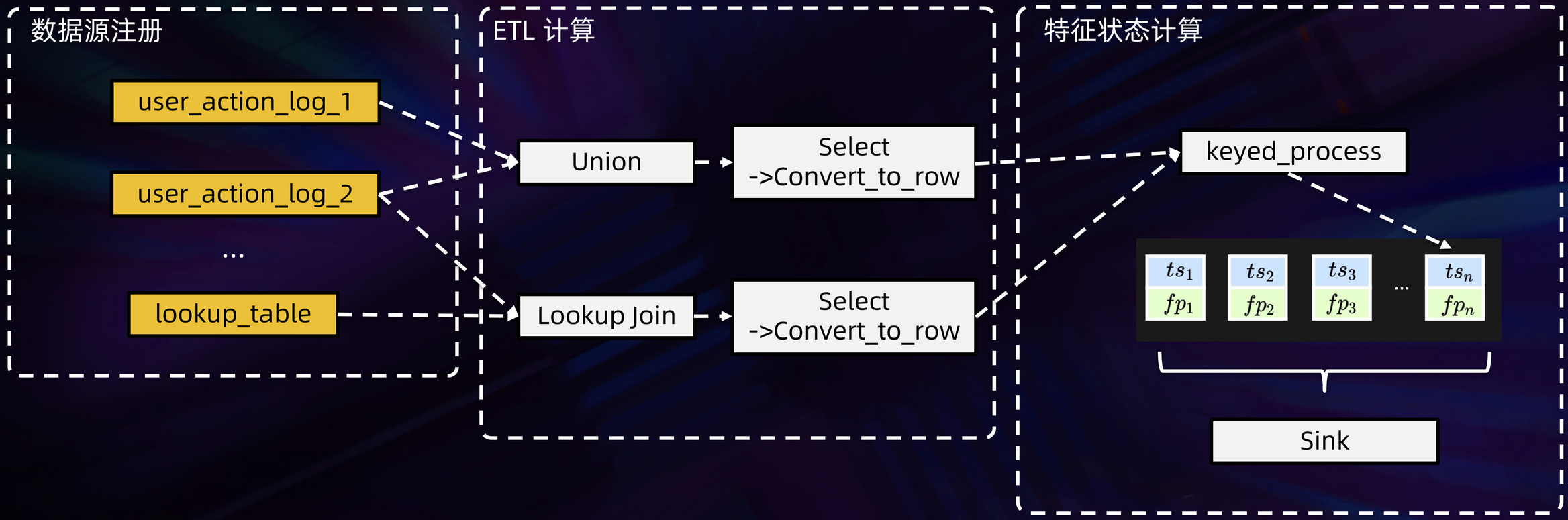
FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

#2 平台架构

Darwin 特征平台

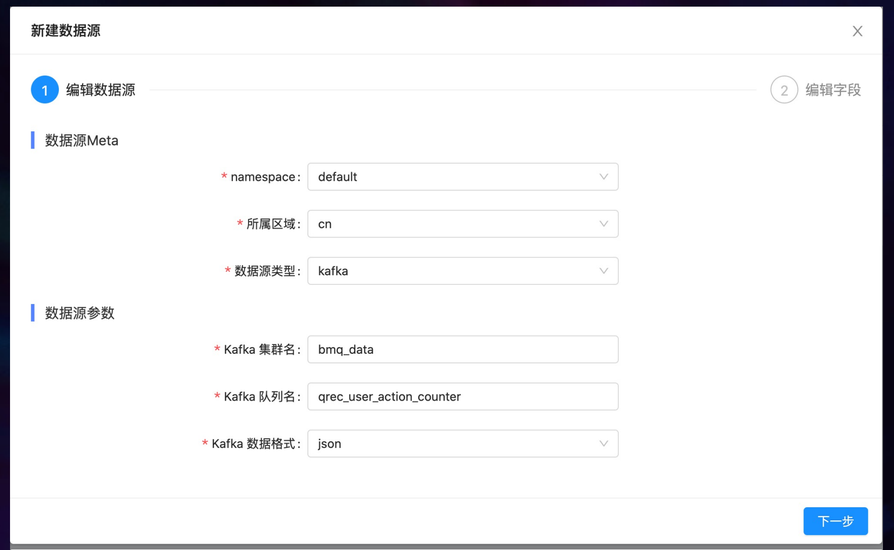

应用场景
抖音推荐
电商推荐
审核风控
产品运营
报表统计
核心服务
特征计算
ETL 计算
在线查询
特征订阅
OLAP 分析
数据源管理
任务管理
监控分析
特征回溯
特征入湖
统一 DSL
ETL 类型特征计算
状态特征计算
计算引擎
Streaming SQL
Batch SQL
Flink CEP
Flink
Spark
存储服务
NoSQL
MySQL
Hbase
Kafka
Hive
Hudi

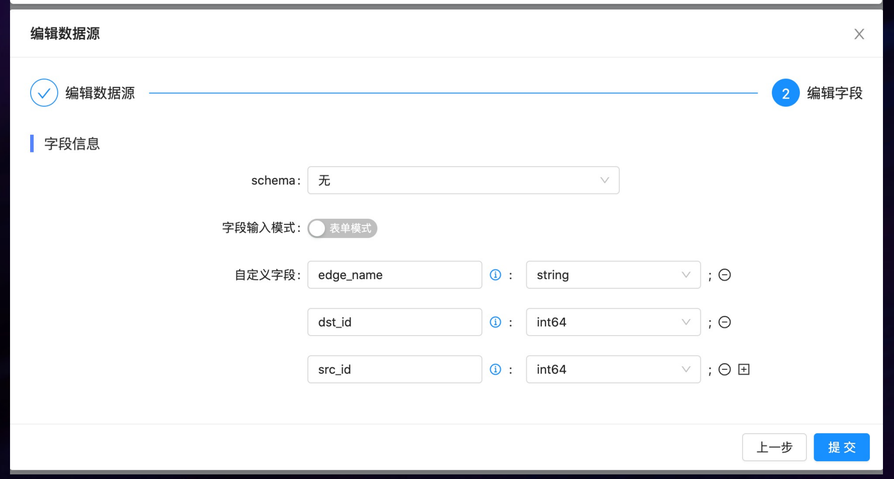
计算模型


数据源注册
user_action_log_1
user_action_log_2
...
lookup_table
ETL 计算
Union
Select
->Convert_to_row
Lookup Join
Select
->Convert_to_row
特征状态计算
keyed_process
ts_1 fp_1
ts_2 fp_2
ts_3 fp_3
...
ts_n fp_n
Sink

计算模型

注册数据源






```
CREATE TABLE qrec_ua_action_source (
    edge_name   varchar,
    dst_id      bigint,
    src_id      bigint,
    source_kafka_ts AS PROCTIME()
) WITH (
    'connector' = 'kafka-0.10',
    'topic' = 'qrec_user_action_counter',
    'properties.group.id' = 'gauss_qrec_user_action_counter_1',
    'properties.cluster' = 'bmq_data_dp',
    'properties.psm' = 'data.gauss.offline',
    'properties.team' = 'Data-arch',
    'parallelism' = '10',
    'properties.owner' = 'xxxxxxx',
    'scan.startup.mode' = 'latest-offset',
    'scan.reset-to-earliest-for-new-partition' = 'false',
    'format' = 'json'
);
```

- 支持 Hive、Kafka、Redis、RPC Service
- 支持 PB、JSON 格式

计算模型

ETL 计算

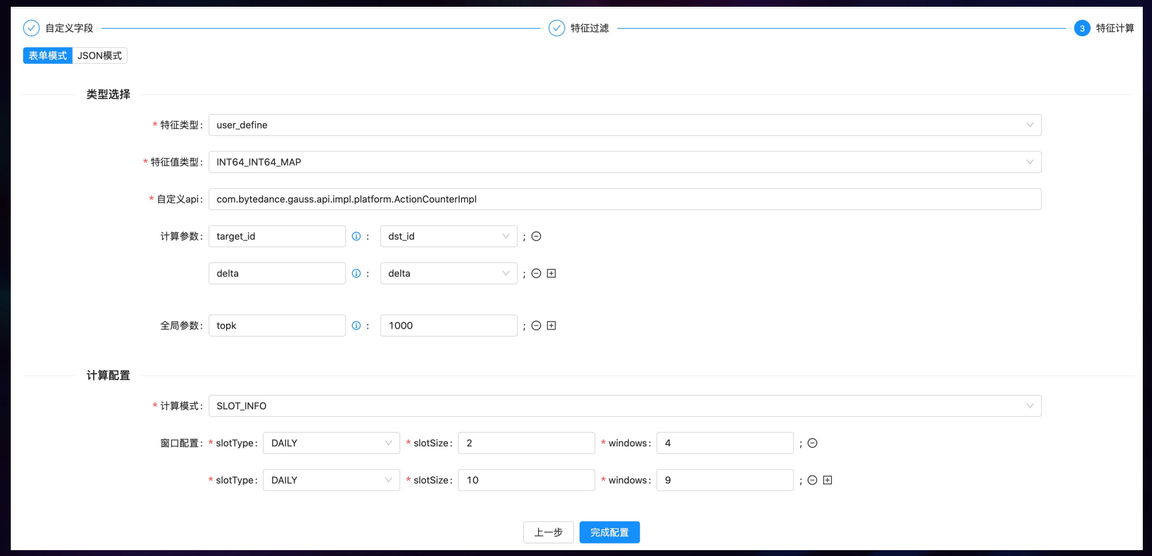



```
CREATE VIEW qrec_ua_action_temp_view AS
SELECT edge_name,
       src_id,
       dst_id,
       CASE
           WHEN (src_id <> 0 AND
                 edge_name in ('gq_ecom_action_1',
                               'gq_ecom_action_2')) THEN 1
           ELSE 0 END AS qrec_ua_action_feature_1,
       CASE
           WHEN (src_id <> 0 AND
                 edge_name in ('gq_ecom_action_3',
                               'gq_ecom_action_4')) THEN 1
           ELSE 0 END AS qrec_ua_action_feature_2
FROM qrec_ua_action_source;
```

- 支持 and、or、==、>=、<=、in、not in 等
- 复杂运算可以通过上传 UDF 表达

计算模型

特征状态计算




```java
public interface GaussFeatureRichFunction<S extends SlotIndexInfo> {

    /**
     * 特征更新接口，上游每条日志会提取必要字段转换为fields，用来更新对应的特征状态
     *
     * @param fields
     *      context: 保存特征名称、主键 和 一些配置参数;
     *      oldFeature: 特征之前的状态
     *      fields: 平台/配置文件 中的抽取字段
     * @return
     */
    FeaturePayload assign(GaussContext context, S slotInfo, FeaturePayload oldFeature, Map<String, Object> fields);

    /**
     * 特征聚合接口，会根据配置的特征计算窗口，读取窗口内所有特征状态，排序后传入该接口
     *
     * @param slotStates, 包含2个field
     *      timeslot: 特征状态对应的时间槽
     *      Feature: 该时间槽的特征状态
     * @return
     */
    FeaturePayload aggregate(GaussContext context, List<Tuple2<S, FeaturePayload>> slotStates);

}
```

- 实现 assign、aggregate 方法来自定义计算逻辑
- 内置了 window counter、二维 counter、序列等特征计算函数

ETL 演进

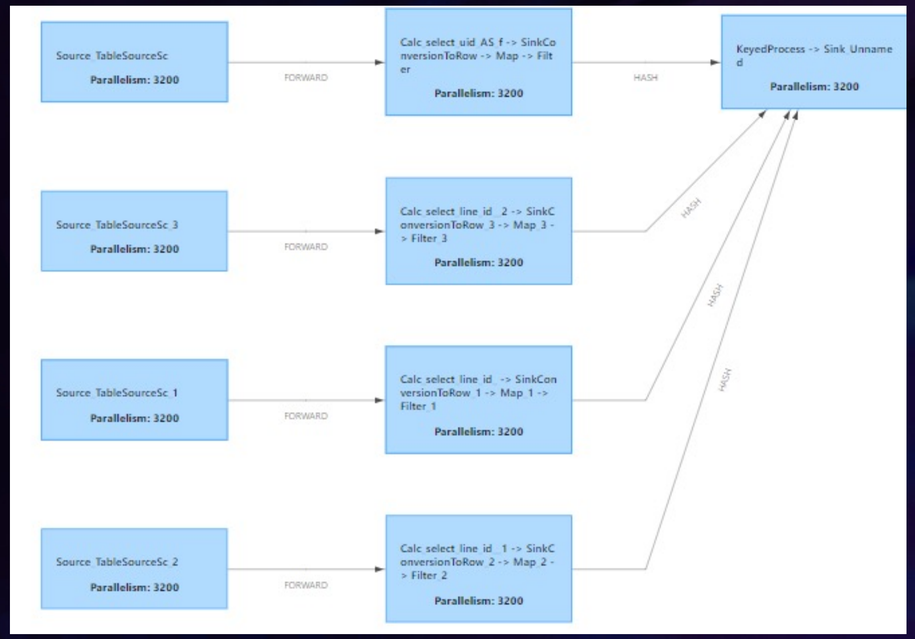
特征-数据源 1 on 1

```sql
CREATE TABLE source_1 (
    source_1_field_1 bigint,
    source_2_field_2 bigint,
    source_kafka_ts AS PROCTIME()
) WITH (
    'connector' = 'xxx',
    'connector.properties' = 'xxx',
);
```

```sql
CREATE VIEW feature_1_view AS
(
    SELECT source_1_field_1,
           source_2_field_2,
    FROM source_1
    WHERE source_1_field_1 IS NOT NULL
      AND source_2_field_2 IS NOT NULL
)
```

```sql
CREATE VIEW feature_2_view AS
(
SELECT source_2_field_3,
       source_2_field_4,
FROM source_1
WHERE source_2_field_3 IS NOT NULL
  AND source_2_field_4 IS NOT NULL
)
```

一个 Source 对应一个特征



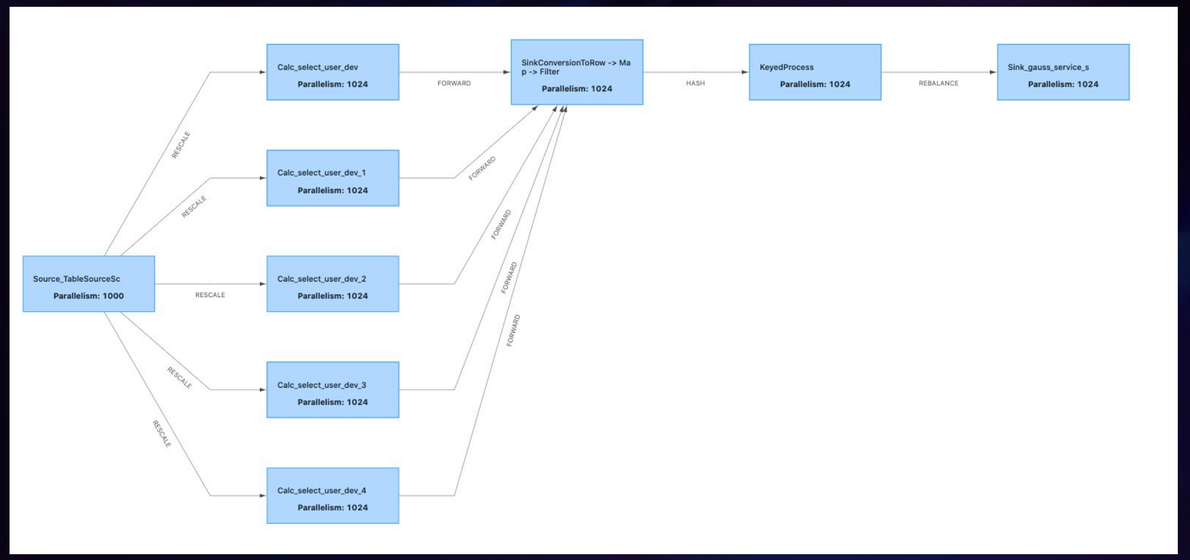
每个 Source 都会初始化 Consumer， 重复消费 Kafka

ETL 演进

特征-数据源 N on 1

```
CREATE VIEW multi_feature_view AS
((
    SELECT source_field_1,
           source_field_2,
           action_type,
           cast(1 AS bigint)  AS delta,
           cast('feature_name_1' AS varchar) AS feature_name
    FROM source_1
    WHERE source_field_1 IS NOT NULL
      AND source_field_2 IS NOT NULL
      AND action_type in ('play')
)
union all
(
    SELECT source_field_1,
           source_field_2
           cast(1 AS bigint)    AS delta,
           cast('feature_name_2' AS varchar) AS feature_name
    FROM source_1
    WHERE source_field_1 IS NOT NULL
      AND source_field_2 IS NOT NULL
      AND action_type in ('show')
))
```

一个 Source 对应多个特征



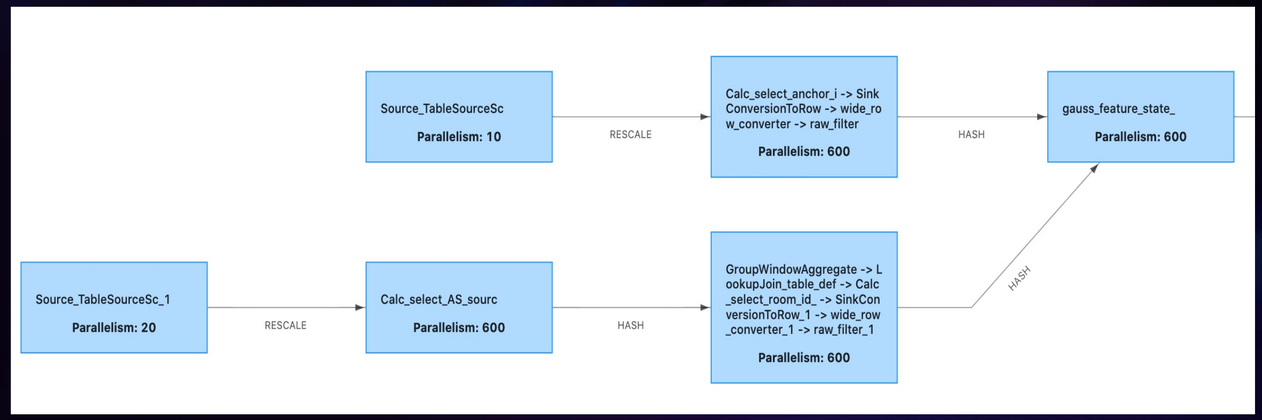
- 通过 union all 的方式将多个数据流合并至一个 source 上
- 不适于特征很多的场景
- 多个特征打散->聚合的过程会有额外开销

ETL 演进

特征-数据源 Unified

```
CREATE VIEW wide_row_1_final_view AS
SELECT
    user_id,
    author_id,
    cast(author_loc as bigint),
    local_life_poi_types,
    local_life_poi_merchants,
    local_life_poi_ids,
    cast(1 as bigint) as delta,
    CASE WHEN action_type=10194 THEN 1 ELSE 0 END AS groupon_click_tag,
    CASE WHEN action_type=10197 THEN 1 ELSE 0 END AS confirm_pag_tag
FROM live_room_index_temp
WHERE CARDINALITY(local_life_poi_types) > 0;
```

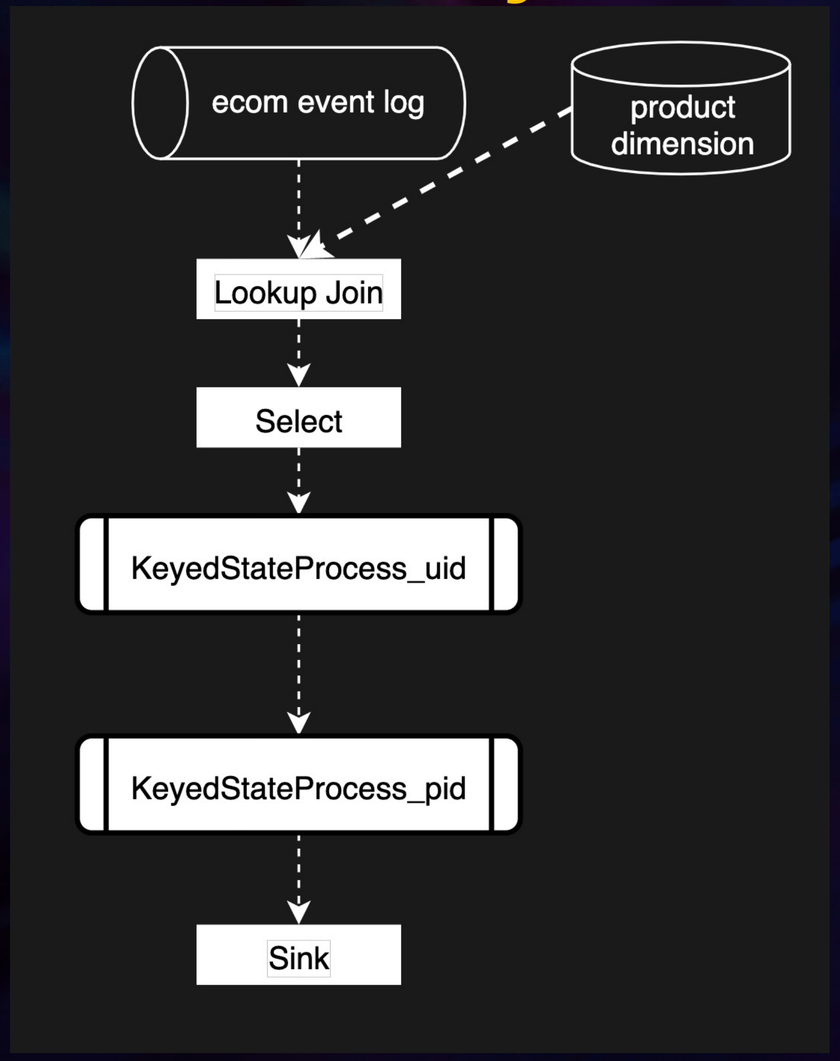
一个 Select 对应多个特征



- CASE WHEN 语法将谓词逻辑拆分到每个单独的特征
- 通过打 tag 的方式，标记该条数据是否属于某个特征

业务场景应用

实际场景-Streaming i2i 计算



场景需求: 计算某个商品的相似度列表

关键点:

- 两次 keyby，先用 uid，计算出于当前商品关联的相似商品；再根据 pid keyby
- 在 State 中维护一个商品相似度列表，持续更新权重

具体步骤:

- Step1. 双流 join，补齐商品类目字段
- Step2. ETL 处理，过滤出属于 click 类的记录
- Step3. 根据 uid 做 keyby， 计算相关物品列表
- Step4. 根据 pid 做 keyby，更新时间分片中相似 item 的权重，聚合计算 item2item 相似度，输出到 online serving

业务调优

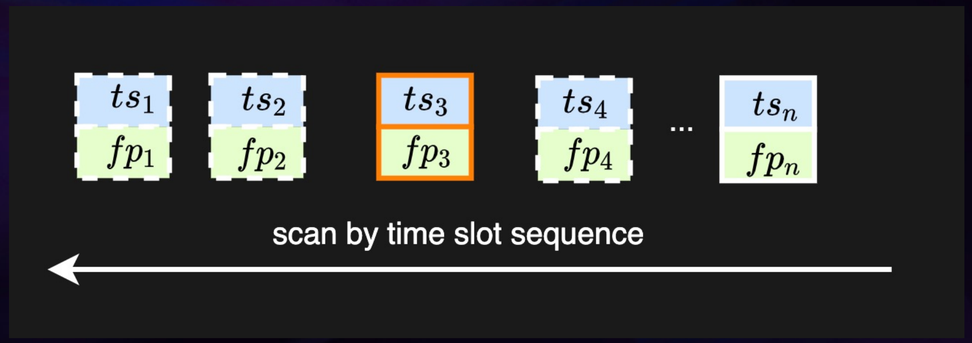
空数据聚合问题

当前 Key Context 下:

MapState< TimeSlotIndex, FeaturePayload>

TimeSlotIndex： long

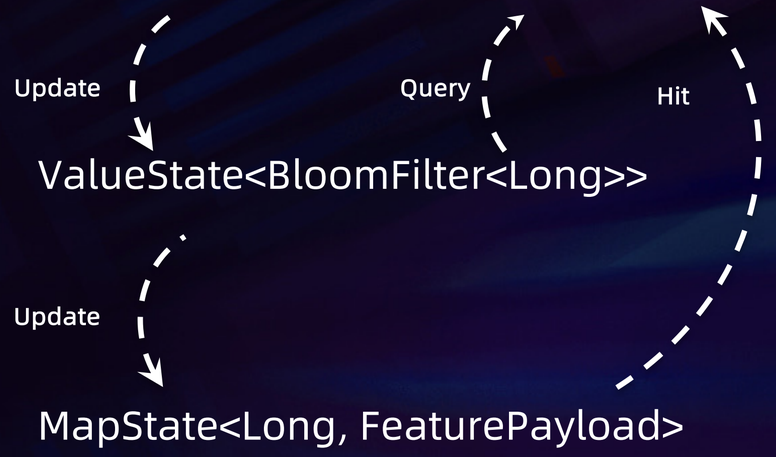
FeaturePayload： pojo



如果数据在时间上分布不均，聚合计算时将读取大量空数据

优化点

CurrentKey:



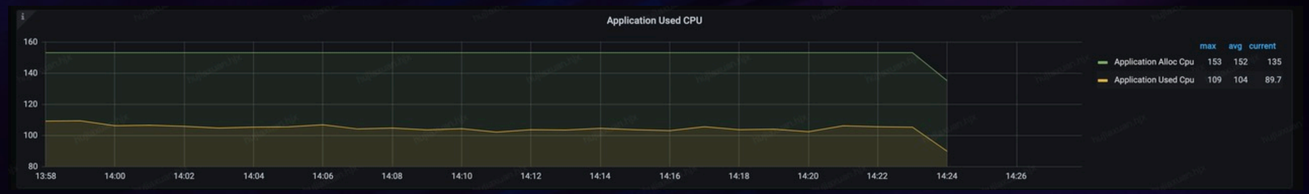
减少对 RocksDB 的查询次数；latency 降低40%

业务调优

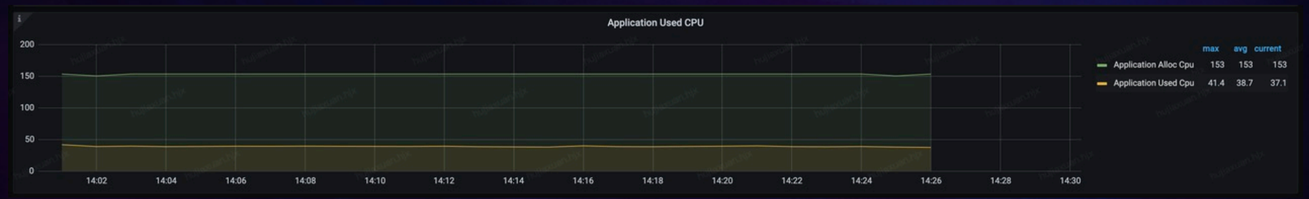
PB 裁剪优化

```
CREATE TABLE src WITH (
    'connector' = 'kafka-0.10',
    'topic' = '[redacted]',
    'properties.cluster' = '[redacted]',
    'properties.group.id' = '[redacted]',
    'scan.manually-commit-offsets-interval' = '5000',
    'scan.startup.mode' = 'group-offsets',
    'properties.prefer.service' = 'dcleader',
    'properties.psm' = 'data.gauss.offline',
    'parallelism' = '5',
    'properties.team' = 'data-arch',
    'properties.owner' = '[redacted]',
    'format' = 'pb',
    'pb.ignore-parse-errors' = 'true',
    'pb.pb-class' = 'com.bytedance.protobuf.mario.MarioEventProto$MarioEvent'
);
```

使用 pb-class 来反序列化 bytes，解析出字段



裁剪未使用的字段后，减少50+的 CPU 使用

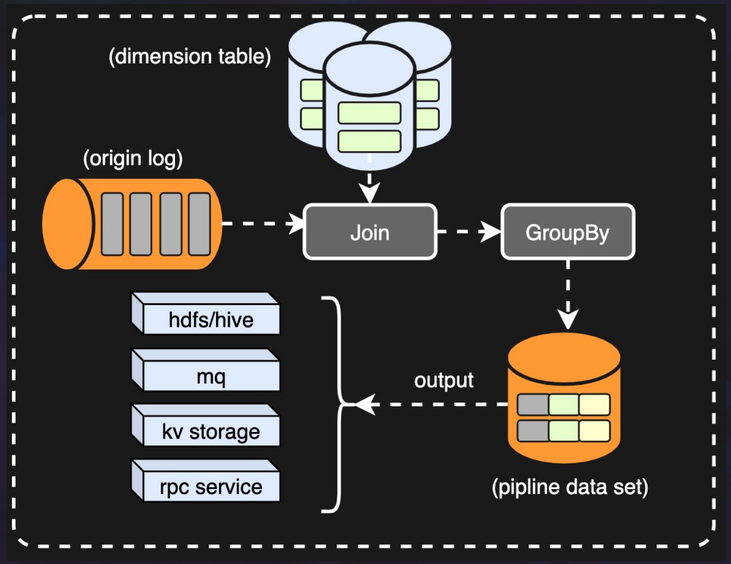


FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

#3 流批一体实践与探索

流批一体探索

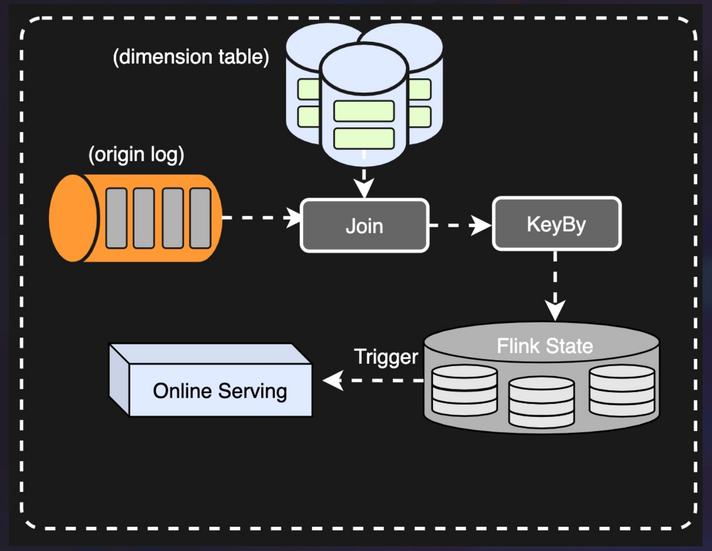
计算一体



核心问题

- 对流处理和批处理分别维护两套代码
- 特征回溯

存储一体



核心问题

- 如何使用历史数据初始化状态
- 通过特征入湖，完成离线数据和实时数据统一

流批一体探索-计算一体

ETL 任务模板

Source

```
CREATE  VIEW hive_src_view AS
SELECT
    source_id,
    target_id,
    ame_cmt_others_cnt_30d,
    ame_cmt_at_others_cnt_30d,
    ame_like_others_videos_cnt_30d,
    ame_enter_others_home_cnt_30d,
    ame_send_msg_to_others_cnt_30d
    `date`
FROM
    hive.data_arch.aweme_intimate_ua_features_history
where
    `date` = '${date}';
```

- 同时支持 unbounded、bounded 数据源

Extract

```
INSERT INTO etl_sink
SELECT  source_id AS primary_key,
        default_secondary_keys() AS secondary_keys,
        merge_count(ame_cmt_others_cnt_30d) as ame_cmt_others_cnt_30d,
        merge_count(ame_cmt_at_others_cnt_30d) as ame_cmt_at_others_cnt_30d,
        merge_count(ame_like_others_videos_cnt_30d) as ame_like_others_videos_cnt_30d,
        merge_count(ame_enter_others_home_cnt_30d) as ame_enter_others_home_cnt_30d,
        merge_count(ame_send_msg_to_others_cnt_30d) as ame_send_msg_to_others_cnt_30d
FROM    hive_src_view
WHERE
    source_id>0
group by
    source_id;
```

- SQL + UDF 可以解决大部分 ETL 需求
- 支持 join 和 union
- 流批一体的执行模式

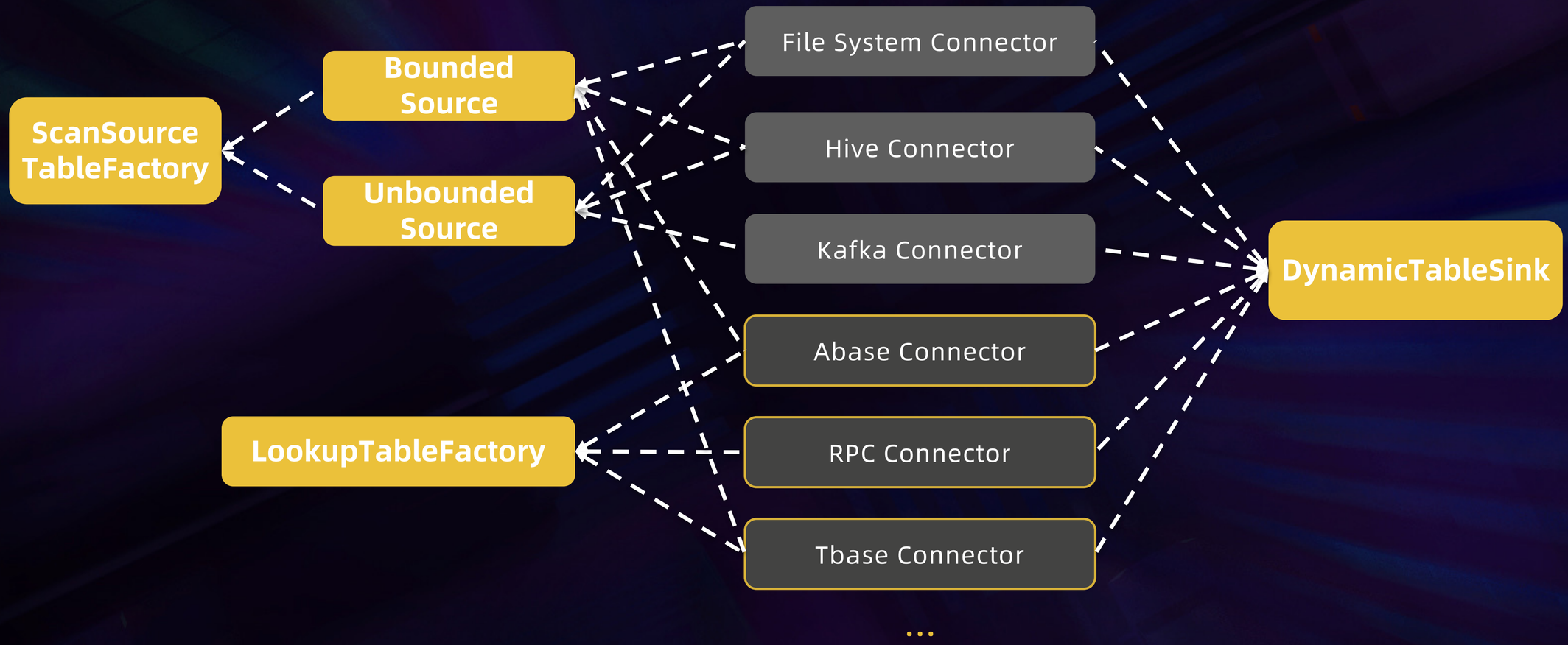
Sink

```
CREATE  TABLE etl_sink(
    primary_key                     BIGINT,
    secondary_keys                  ARRAY<BIGINT>,
    ame_cmt_others_cnt_30d              MAP<BIGINT, BIGINT>,
    ame_cmt_at_others_cnt_30d           MAP<BIGINT, BIGINT>,
    ame_like_others_videos_cnt_30d      MAP<BIGINT, BIGINT>,
    ame_enter_others_home_cnt_30d       MAP<BIGINT, BIGINT>,
    ame_send_msg_to_others_cnt_30d      MAP<BIGINT, BIGINT>
)
WITH (
    'connector' = 'darwin-etl',
    'feature-group' = 'aweme_intimate_features_1',
    'sink-type' = 'gauss',
    'psm' = 'data.gauss.master.service.hl',
    'caller' = 'data.darwin_offline.etl_sink_hive',
    'batch-size' = '100',
    'expire-seconds' = '604800'
);
```

- 自定义 sink connectors
- Table API 支持 Schema 解析

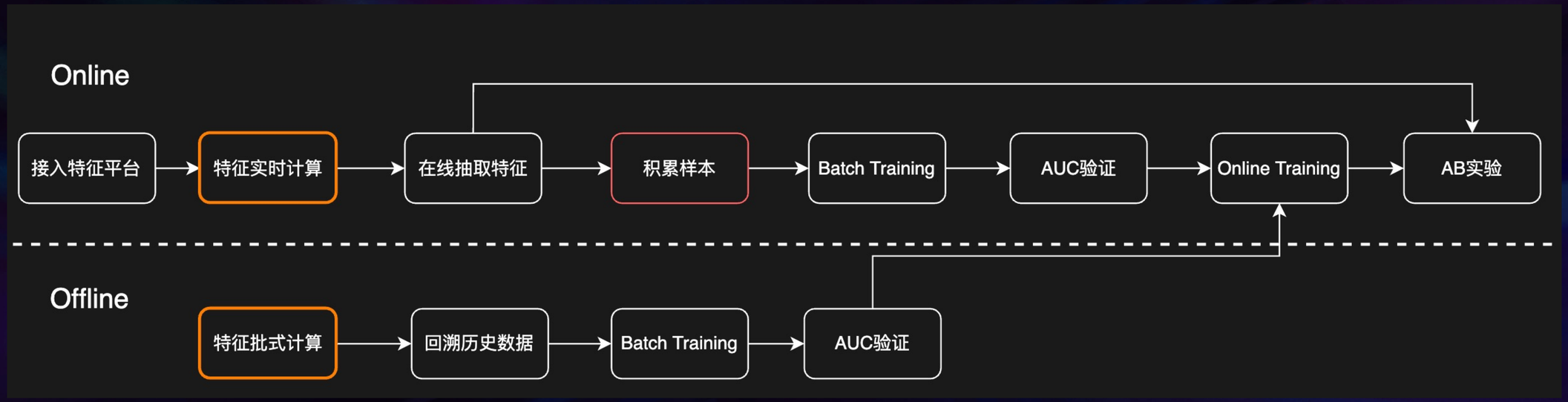
流批一体探索-计算一体

流批一体 Connectors



流批一体探索-计算一体

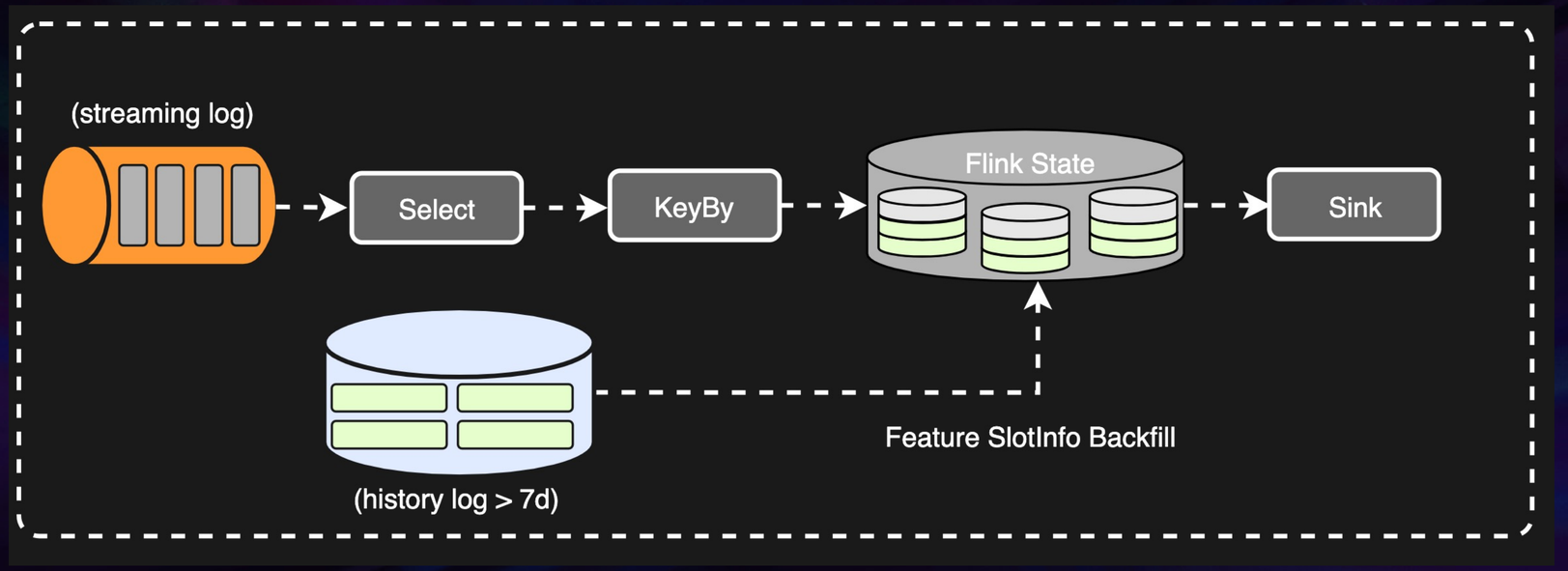
特征回溯



- Flink SQL 的流批一体能力，同一份代码既可以实时计算，又可以批式计算
- 节省开发成本，加速特征迭代过程

流批一体探索-存储一体

状态初始化



核心需求

特征上线或者迭代后，需要积累历史数据

任务 DAG 改变后，仍然可以恢复状态

解决方案

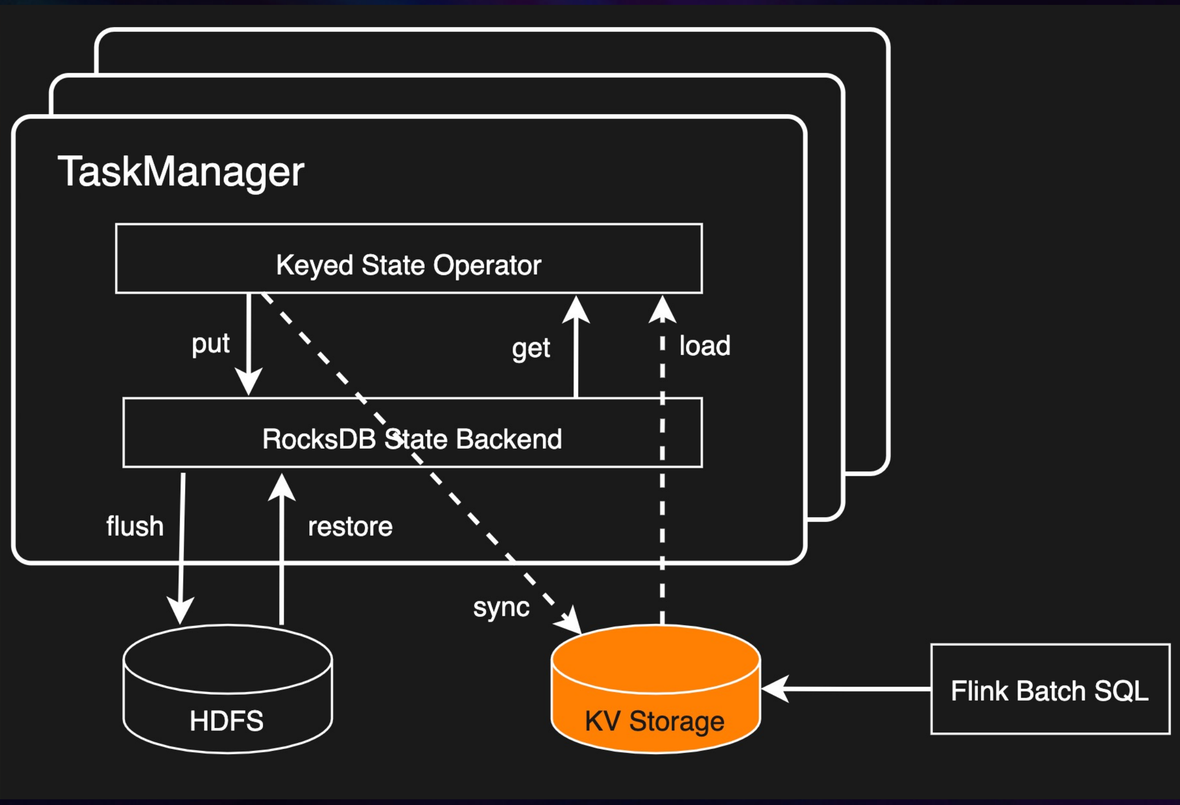
外置 StateBackend

双数据源消费

流批一体探索-存储一体

状态初始化

外置 StateBackend



Solution:

- 使用 KV 服务存储状态数据
- 预先批处理计算好特征历史数据，刷入 KV 服务
- 任务启动时先读取历史数据，回填 State
- 本地状态和外置状态保持同步

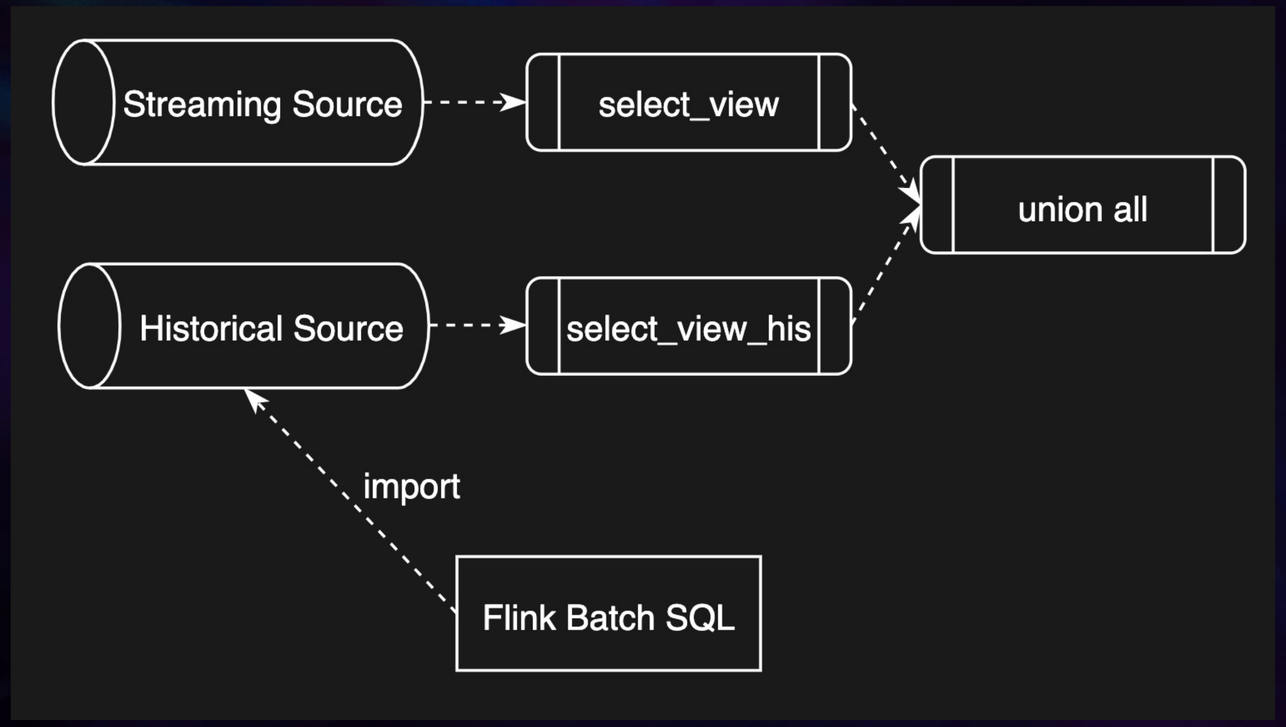
Crons:

- 额外的存储成本
- 本地 State 和外置 State 的一致性问题

流批一体探索-存储一体

状态初始化

双数据源消费



Solution:

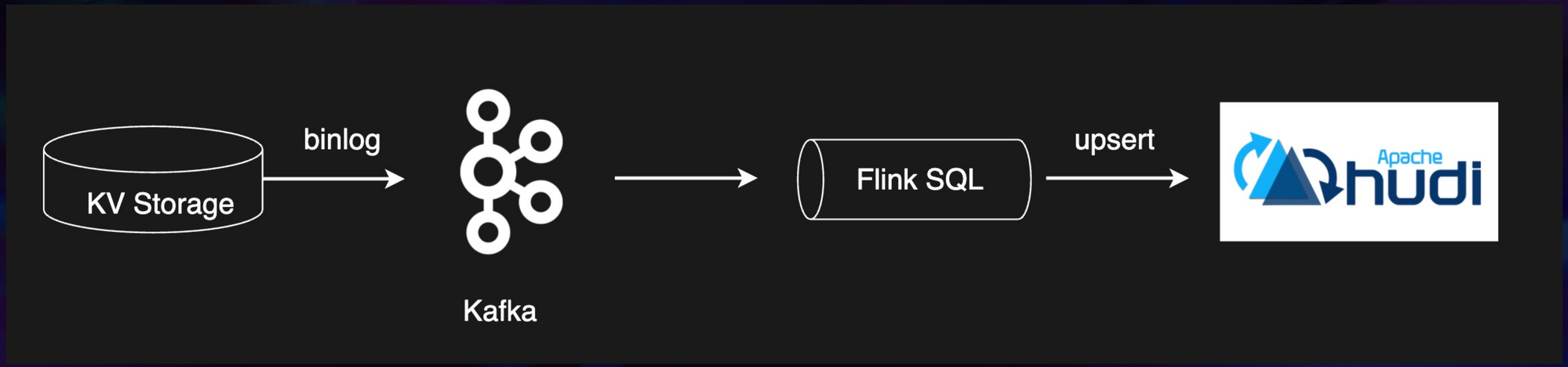
- Finish 任务无法完成 Checkpoint
- 创建一个额外的 MQ，批处理任务将历史数据回填进去
- 历史数据消费完成后再开始触发计算

Crons:

- 流批割裂，需要多维护一个批处理任务
- 历史消费完成后，task 仍然一直运行
- 不适合保序场景

流批一体探索-存储一体

特征入湖



传统数仓：

- T+1 更新，或者使用 Lambda 架构加速
- 保存快照表、增量表，两份数据
- 不支持索引

Hudi 方案：

- 近实时更新，数据可见
- 保存 record 级别的变更数据，支持快照读、增量读
- 支持多种索引

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

#4 未来规划

未来规划

- 全面迁移 Flink SQL，推进 SQL + UDF 的使用方式覆盖全场景
- 部分场景牺牲实时性，动态调整 Timer 触发时间，流量打平到全天，换取潮汐资源利用率
- 提高自动化运维程度，实时采集 CPU、内存、Lag size 等指标，计算理想资源分配值，动态调整


FLINK FORWARD
#ASIA 2021
ONLINE
实时即未来
REAL-TIME IS THE FUTURE
THANKS